C++ DEFINITIONS GUIDE FOR ROBOTICS AND CHIP PROGRAMMING

This comprehensive reference document contains definitions, data structures, syntax templates, hardware mapping constants, and key terminology used in C++ chip programming (microcontrollers) and robotics software design.

TABLE OF CONTENTS

1. GLOSSARY OF CORE EMBEDDED & ROBOTICS TERMINOLOGY
2. C++ KEYWORDS & QUALIFIERS DEFINITIONS
3. FIXED-WIDTH DATA TYPES & MEMORY DEFINITIONS
4. HARDWARE REGISTER & BIT MANIPULATION DEFINITIONS
5. PERIPHERAL CONFIGURATION DEFINITIONS (GPIO, PWM, ADC, BUSES)
6. INTERRUPT SERVICE ROUTINES & TIMER DEFINITIONS
7. ROBOTICS KINEMATICS & CONTROL DATA STRUCTURES
8. HARDWARE ABSTRACTION LAYER (HAL) CLASS TEMPLATES
9. NON-ALLOCATING DATA STRUCTURE DEFINITIONS

1. GLOSSARY OF CORE EMBEDDED & ROBOTICS TERMINOLOGY

* ADC (Analog-to-Digital Converter):
  A hardware peripheral that samples continuous physical voltages (e.g., sensor outputs) and converts them into discrete integer values.

* Bare-Metal Programming:
  Developing firmware directly for hardware registers without an underlying Operating System (OS) or Real-Time Operating System (RTOS).

* Bitmask:
  A binary pattern of bits used with bitwise logical operators (AND, OR, XOR) to inspect, set, or clear specific bits within a hardware control register.

* Bus:
  A shared communication pathway used to transfer data between chips, peripherals, or processors (e.g., I2C, SPI, CAN, UART).

* Closed-Loop Control:
  A system that uses continuous sensor feedback to adjust actuator outputs and minimize the error between target state and measured state (e.g., PID loop).

* Deadband:
  A range of input values where an actuator output is intentionally set to zero to prevent motor jitter around the neutral position.

* Duty Cycle:
  The percentage of time a Pulse Width Modulation (PWM) signal remains in a
  HIGH voltage state over one full cycle period.

* Encoder:
  A sensor attached to a motor shaft that generates electrical pulses to measure
  rotational angle, angular velocity, and direction.

* FSM (Finite State Machine):
  A behavioral logic pattern where a system exists in exactly one state at any
  given time and transitions between states based on inputs and conditions.

* GPIO (General Purpose Input/Output):
  Digital silicon pins on a microcontroller that can be dynamically configured
  as inputs (reading digital levels) or outputs (driving voltages).

* HAL (Hardware Abstraction Layer):
  A software module that hides low-level hardware register details behind a
  clean, high-level C++ class or function interface.

* ISR (Interrupt Service Routine):
  A dedicated function executed immediately by the CPU hardware when a specific
  hardware event or signal occurs, pausing normal execution flow.

* Kinematics:
  The branch of mechanics describing the motion of robotic links and joints
  without considering the forces that cause the motion (Forward & Inverse).

* Memory-Mapped I/O (MMIO):
  A hardware architecture where physical peripheral control registers are mapped
  to specific numeric addresses within the chip's unified memory space.

* Open-Loop Control:
  A system that sends commands to actuators without measuring or reacting to
  the actual resulting output or external disturbances.

* Polling:
  A software technique where the CPU continuously checks the status of a pin or
  register in a loop to detect state changes.

* Prescaler:
  A hardware clock divider circuit that slows down the primary CPU clock frequency
  to drive timers at lower, manageable rates.

* PWM (Pulse Width Modulation):
  A technique for simulating continuous analog outputs by toggling a digital
  pin HIGH and LOW at high frequencies with varying duty cycles.

* Register:
  A small, extremely fast memory location built directly into silicon to control

chip configurations, inspect statuses, or read/write data bytes.

* RTOS (Real-Time Operating System):
  An OS designed for deterministic response times, managing multi-task scheduling, queues, and mutexes with guaranteed timing constraints.

* UART (Universal Asynchronous Receiver-Transmitter):
  A point-to-point asynchronous serial protocol using separate Transmit (TX) and Receive (RX) lines without a shared clock wire.

2. C++ KEYWORDS & QUALIFIERS DEFINITIONS

In embedded C++, specific keywords dictate memory alignment, compiler optimizations, and runtime behavior:

* 'volatile':
  Tells the compiler that a variable's value can be changed by external hardware or ISRs outside the active code flow. Prevents compiler optimizations (such as caching variables in CPU registers or removing seemingly idle loops).

  Definition Example:
```
volatile uint32_t* const SYSTEM_TIMER_REG = (uint32_t*)0x40001000;
```

* 'constexpr':
  Specifies that a value or function can be evaluated at compile-time rather than runtime. Saves execution time and flash memory on microcontrollers.

  Definition Example:
```
constexpr float WHEEL_RADIUS_METERS = 0.045f;
constexpr uint32_t SEC_TO_MS(uint32_t sec) { return sec * 1000; }
```

* 'inline':
  Suggests to the compiler to replace function calls directly with the function's body code, eliminating function call overhead (stack pushing/popping) at the cost of slightly larger code size.

* 'static':
  - Inside functions: Retains variable state across function calls without allocating memory on the global heap.
  - At file scope: Restricts symbol visibility strictly to the current file (translation unit).

* 'noexcept':
  Declares that a function will never throw exceptions. Critical in real-time embedded programming, allowing the compiler to optimize out exception-unwinding code.

* 'explicit':
  Prevents implicit conversions or copy-initialization of C++ constructors,
  ensuring hardware parameters are explicitly converted.

* 'alignas' & 'alignof':
  Specifies precise memory alignment rules for data structures, ensuring hardware
  Direct Memory Access (DMA) controllers can read buffers correctly.

==
3. FIXED-WIDTH DATA TYPES & MEMORY DEFINITIONS
==

Standard C++ data types (like 'int' or 'long') have platform-dependent sizes.
Embedded code relies on fixed-width types from <cstdint> and <cstddef>:

Type Name   Bit Size   Byte Size   Range / Usage
----
int8_t   8-bit   1 byte   -128 to 127
uint8_t   8-bit   1 byte   0 to 255 (Bytes, 8-bit registers, flags)
int16_t   16-bit   2 bytes   -32,768 to 32,767
uint16_t   16-bit   2 bytes   0 to 65,535 (10/12-bit ADC readings)
int32_t   32-bit   4 bytes   -2,147,483,648 to 2,147,483,647
uint32_t   32-bit   4 bytes   0 to 4,294,967,295 (Ticks, Timers,
Addresses)
float   32-bit   4 bytes   Single-precision IEEE 754 floating point
double   64-bit   8 bytes   Double-precision (Avoid on 8/32-bit MCUs
without FPU)
uintptr_t   Native   Platform   Unsigned integer guaranteed to fit a pointer
size_t   Native   Platform   Unsigned integer representing object memory
size

==
4. HARDWARE REGISTER & BIT MANIPULATION DEFINITIONS
==

To write clean register-level firmware, standard bit manipulation macros or inline
C++ templates are defined.

A. Standard Bitwise Macros

```
#ifndef EMBEDDED_BIT_MACROS_H
#define EMBEDDED_BIT_MACROS_H

#define BIT(x)                 (1UL << (x))
#define SET_BIT(reg, bit)      ((reg) |= BIT(bit))
#define CLEAR_BIT(reg, bit)    ((reg) &= ~BIT(bit))
```

```
#define TOGGLE_BIT(reg, bit)   ((reg) ^= BIT(bit))
#define READ_BIT(reg, bit)     (((reg) >> (bit)) & 0x01U)
#define WRITE_BIT(reg, bit, val) \
    ((val) ? SET_BIT(reg, bit) : CLEAR_BIT(reg, bit))

#endif
```

B. Type-Safe C++ Register Wrapper Template

```
#include <cstdint>

template <uintptr_t Address>
struct Register32 {
    static void write(uint32_t value) {
        *reinterpret_cast<volatile uint32_t*>(Address) = value;
    }

    static uint32_t read() {
        return *reinterpret_cast<volatile uint32_t*>(Address);
    }

    static void setBit(uint8_t bit) {
        *reinterpret_cast<volatile uint32_t*>(Address) |= (1UL << bit);
    }

    static void clearBit(uint8_t bit) {
        *reinterpret_cast<volatile uint32_t*>(Address) &= ~(1UL << bit);
    }

    static bool isBitSet(uint8_t bit) {
        return (*reinterpret_cast<volatile uint32_t*>(Address) & (1UL << bit)) !=
0;
    }
};
```

5. PERIPHERAL CONFIGURATION DEFINITIONS (GPIO, PWM, ADC, BUSES)

A. General Purpose Input/Output (GPIO) Definitions

```
enum class PinMode : uint8_t {
    INPUT_FLOATING,
    INPUT_PULLUP,
    INPUT_PULLDOWN,
    OUTPUT_PUSHPULL,
    OUTPUT_OPENDRAIN,
    ALTERNATE_FUNCTION
};
```

```
enum class PinState : uint8_t {
    LOW = 0,
    HIGH = 1
};

struct GpioConfig {
    uint8_t port;
    uint8_t pin;
    PinMode mode;
    PinState initial_state;
};

B. Pulse Width Modulation (PWM) Definitions
-------------------------------------------------------------------------------
struct PwmConfig {
    uint32_t frequency_hz;   // e.g., 50 Hz for Servos, 20,000 Hz for DC Motors
    uint16_t resolution_bits; // e.g., 8-bit (0-255), 12-bit (0-4095)
    float duty_cycle_percent; // 0.0f to 100.0f %
};

C. Communication Bus Definitions
-------------------------------------------------------------------------------
// UART Baud Rates
enum class BaudRate : uint32_t {
    BAUD_9600   = 9600,
    BAUD_115200 = 115200,
    BAUD_921600 = 921600
};

// I2C Speed Modes
enum class I2CSpeed : uint32_t {
    STANDARD_100KHZ = 100000,
    FAST_400KHZ     = 400000,
    HIGH_3_4MHZ     = 3400000
};

// SPI Clock Polarity and Phase
enum class SpiMode : uint8_t {
    MODE_0 = 0, // CPOL=0, CPHA=0
    MODE_1 = 1, // CPOL=0, CPHA=1
    MODE_2 = 2, // CPOL=1, CPHA=0
    MODE_3 = 3  // CPOL=1, CPHA=1
};


===============================================================================
6. INTERRUPT SERVICE ROUTINES & TIMER DEFINITIONS
===============================================================================
```

Interrupts interrupt main code loop based on trigger events. Shared data must use atomic operations or volatile pointers.

A. Hardware Interrupt Vector Definitions

```
enum class InterruptTrigger : uint8_t {
    RISING_EDGE,
    FALLING_EDGE,
    CHANGE_EDGE,
    LOW_LEVEL,
    HIGH_LEVEL
};

typedef void (*IsrCallback)(void);

struct ExternalInterruptConfig {
    uint8_t pin;
    InterruptTrigger trigger;
    IsrCallback callback;
};
```

B. Timer & Encoder Tick Counter Definitions

```
struct EncoderTicks {
    volatile int32_t left_counter;
    volatile int32_t right_counter;
    volatile uint32_t last_timestamp_us;
};
```

7. ROBOTICS KINEMATICS & CONTROL DATA STRUCTURES

A. 2D Pose and Velocity Definitions (Mobile Robots)

```
struct Vector2D {
    float x;
    float y;
};

struct Pose2D {
    float x;     // X position in meters
    float y;     // Y position in meters
    float theta; // Orientation heading in radians (-PI to +PI)
};

struct Twist2D {
    float linear_x;  // Forward velocity in m/s
    float linear_y;  // Lateral velocity in m/s (0 for differential drive)
```

```
    float angular_z; // Angular turn rate in rad/s
};

B. Differential Drive Wheel Velocities
--------------------------------------------------------------------------------
struct WheelSpeeds {
    float left_rad_sec;
    float right_rad_sec;
};

// Forward Kinematics Calculation Helper Struct
struct RobotPhysicalParams {
    float wheel_radius_m;
    float track_width_m;  // Distance between left and right wheels
    uint16_t encoder_cpr; // Ticks per motor revolution
};

C. PID Controller Gains Struct
--------------------------------------------------------------------------------
struct PidGains {
    float kp; // Proportional Gain
    float ki; // Integral Gain
    float kd; // Derivative Gain
    float output_min; // Actuator lower limit (e.g., -100% PWM)
    float output_max; // Actuator upper limit (e.g., +100% PWM)
};


================================================================================
8. HARDWARE ABSTRACTION LAYER (HAL) CLASS TEMPLATES
================================================================================

In C++ robotics, pure virtual interfaces (abstract base classes) allow swapping
underlying hardware implementations without changing higher-level robot logic.

A. Abstract Motor Driver Interface
--------------------------------------------------------------------------------
class IMotorDriver {
public:
    virtual ~IMotorDriver() = default;
    virtual void init() = 0;
    virtual void setSpeed(float percent) = 0; // Range: -100.0f to +100.0f
    virtual void stop() = 0;
};

B. Abstract Distance Sensor Interface
--------------------------------------------------------------------------------
class IDistanceSensor {
public:
    virtual ~IDistanceSensor() = default;
```

```
    virtual bool init() = 0;
    virtual float readDistanceMeters() = 0;
};


================================================================================
9. NON-ALLOCATING DATA STRUCTURE DEFINITIONS
================================================================================

To strictly comply with zero dynamic memory allocation policies on embedded
chips, fixed-capacity static ring buffers (circular queues) are defined.

--------------------------------------------------------------------------------
#include <cstddef>
#include <array>

template <typename T, size_t Capacity>
class StaticRingBuffer {
private:
    std::array<T, Capacity> buffer;
    size_t head = 0;
    size_t tail = 0;
    size_t count = 0;

public:
    bool push(const T& item) {
        if (count >= Capacity) return false; // Buffer full
        buffer[head] = item;
        head = (head + 1) % Capacity;
        count++;
        return true;
    }

    bool pop(T& out_item) {
        if (count == 0) return false; // Buffer empty
        out_item = buffer[tail];
        tail = (tail + 1) % Capacity;
        count--;
        return true;
    }

    bool isFull() const { return count == Capacity; }
    bool isEmpty() const { return count == 0; }
    size_t size() const { return count; }
    constexpr size_t max_capacity() const { return Capacity; }
};

================================================================================
                            END OF DEFINITIONS GUIDE
================================================================================
```